

Assignment Theme:

“Campus Portal Mini-System” Context: You are tasked to develop a mini “Campus Portal” where different modules (Students, Courses, and Announcements) will be managed. The goal is to understand how to build loosely coupled components using Spring Boot’s IoC and DI and consume them later in a React front end.

Part 1 – Spring Boot Core (IoC + DI)

Task 1: Basic Service Injection

Objective: Demonstrate dependency injection through interfaces and beans.

Instructions:

1. Create an interface `MessageService` with a method `String getMessage()`.
2. Implement two beans:
 - o `StudentMessageService` → returns "Welcome Student to Campus Portal".
 - o `AdminMessageService` → returns "Welcome Admin to Campus Portal".
3. Inject these services into a `WelcomeController` using:
 - o **Field injection**
 - o **Constructor injection**
 - o **Setter injection**
4. Configure which bean should load using `@Primary` or `@Qualifier`.

 **Learning Outcome:** Understand different injection types and Spring’s IoC container behavior.

Task 2: Layered Architecture with DI

Objective: Practice service-to-service injection.

Instructions:

1. Create:
 - a. `StudentService` – handles student logic.
 - b. `CourseService` – handles course logic.
2. Both depend on a common `LoggerService` (with a `log (String message)` method).
- 3.
4. Demonstrate loose coupling by injecting `LoggerService` into both services.
5. Create a simple REST controller `/api/info` that calls these services and returns a combined JSON response like:

```
{  
  "student": "Student Service Active",  
  "course": "Course Service Active",  
  "log": "Logged successfully"  
}
```

✅ Learning Outcome: Show IoC wiring in a multi-layer architecture.

Part 2 – Frontend (React JS)

Task 3: Simple React Front Display

Objective: Fetch data from the backend IoC example.

Instructions:

1. Create a React app with a component.
2. On mount, fetch from /api/info (use fetch or axios).
3. Display each JSON key/value in a styled grid using Bootstrap.
4. Add a button “Refresh Info” to re-fetch and update state.

✅ Learning Outcome: Connecting React with a Spring Boot backend.

Part 3 – Advanced DI Practice (Optional Bonus)

Task 4: Profile-Based Bean Injection

Objective: Show how Spring activates different beans based on environment.

Instructions:

1. Create two implementations of DatabaseConfig:
 - a. MySQLDatabaseConfig (active in dev profile)
 - b. PostgreDatabaseConfig (active in prod profile)
2. Each returns a string with database info.
3. Expose an endpoint /api/dbinfo that tells which database config is currently active.

✅ Learning Outcome: Understand environment-specific bean wiring.

Part 4 – Consolidated Practical Problem

Task 5: Campus Notification System

Objective: Combine IoC + DI + REST + React.

Scenario: Admin can trigger a “Campus Notice” that shows on the React front page.

Backend Steps:

1. Create NotificationService and inject it into NotificationController.
2. Store notices temporarily in a list (no DB yet).
3. Endpoints:
 - a. POST /api/notice – add a new notice
 - b. GET /api/notice – list all notices
4. Use @Autowired or constructor DI for all services.

Frontend Steps:

1. Create NoticeBoard component in React.
2. Display all notices in cards.
3. Provide a small form (title + message) to post a new notice (using POST).
4. Use state and re-render on submit.

 Learning Outcome: Practical application of IoC and DI through full request flow.

Submission Deliverables

Deliverable	Description
Backend	Spring Boot project (src folder zipped) showing IoC + DI tasks
Frontend	React app showing data fetch and rendering
Report	Short PDF/Word explaining injection types used, class relationships, and screenshots of working results

Optional Extension (for Next Week) Add Spring Data JPA to persist notices in a database.

Add Spring Security to restrict POST endpoints to admin users only.

Web Engineering (CSC-324)

Assignment: Assignment 2

Date: October 21, 2025,

Instructor: Syed Nouman Ali Shah

Max Marks: **50**

Submission Date: November 10, 2025

Integrate JWT later.